Kafka tutorial deck: Stream Processing with Kafka

Local Kafka + Kafka UI + CLI (Python) | ~1.5 h

What this deck is

This deck is a **guided tutorial**.

We will:

- start Kafka + Kafka UI locally
- create and inspect topics (partitions, replication)
- produce JSON events and consume them
- show **offsets, replay, consumer groups**
- connect each concept to a concrete command

Mental model in 60 seconds

Kafka is a **distributed, append-only log**.

- **Topics**: named logs
- **Partitions**: parallel logs inside a topic (ordering is per-partition)
- **Offsets**: positions in a partition
- **Consumer groups**: how we scale reads + how we control replay

Demo environment (what runs where)

We run everything via Docker Compose:

- `zookeeper` — coordination for this local setup
- `kafka` — broker (2 listeners)
- **internal**: `kafka:29092` (for other containers + CLI exec)
- **external**: `localhost:9092` (for host tools, if needed)
- `kafka-ui` — web UI at `http://localhost:8089`

Tools we use (repo root)

- `setup.py` — verify prerequisites (Docker, Compose, ports)
- `up.py` — start containers and wait for Kafka readiness
- `down.py` — stop + remove containers/volumes (clean reset)
- `topic.py` — create/list/describe/delete topics
- `produce.py` — send JSON events to a topic
- `consume.py` — consume events with a consumer group

Step 1 — Setup check

```
python3 setup.py --check-ports
```

Step 2 — Start Kafka + UI

```
python3 up.py --reset
```

Then open:

- `http://localhost:8089`

Under the hood: readiness check

`up.py` waits until Kafka answers to:

```
kafka-topics --bootstrap-server kafka:29092 --list
```

Step 3 — Create a demo topic (3 partitions)

```
python3 topic.py create demo-events
```

Why 3 partitions:

- it gives us a real scaling constraint later (max 3 active consumers per group)

Step 4 — Inspect the topic

```
python3 topic.py describe demo-events
```

Point at:

- partitions (0..2)
- leader broker
- ISR (in-sync replicas)

Step 5 — Start a consumer (group A)

```
python3 consume.py demo-events --group demo-group --from-beginning
```

Step 6 — Produce JSON events

```
python3 produce.py demo-events --count 10 --sleep 0.3
```

Offsets and replay (concept)

- offsets are tracked **per partition, per group**
- a new group can replay from the beginning

Replay demo (new group id)

```
python3 consume.py demo-events --group demo-group-2 --from-beginning
```

Consumer group split (real scaling behavior)

Run 2 consumers in the same group:

```
python3 consume.py demo-events --group demo-group-split --instances 2 --print-meta --from-beginning
```

Then produce a few events:

```
python3 produce.py demo-events --count 12 --sleep 0.1
```

What you should see:

- `[c1]`, `[c2]` prefixes
- different `Partition:` values per consumer

Clean reset

```
python3 down.py
```

Production differences (high level)

- replication factor > 1 (multi-broker)
- security (SASL/TLS)
- schema registry / contracts
- monitoring: consumer lag, broker health, throughput
- careful topic design (partitions, retention, compaction)